

M.A.T.E.

MEETING ADVENTURE TRAVEL ESCAPE

COMPLETE PROJECT DOCUMENTATION

Project Proposal • Business Plan • PRD • Technical Architecture
Security • Budget • Roadmap

Version 1.0 | 9 October 2026

Index

Document Purpose and Scope . . . 3
How to use this document . . . 3

Executive Summary . . . 3
Recommended launch focus . . . 3

1. Project Proposal . . . 4
1.1 Problem statement . . . 4
1.2 Proposed solution . . . 4
1.3 Product objectives . . . 4
1.4 Target users . . . 4
1.5 Value proposition . . . 4

2. Business Plan and Go-to-Market . . . 4
2.1 Positioning . . . 4
2.2 Customer acquisition strategy . . . 5
2.3 Launch phases . . . 5
2.4 North Star and key metrics . . . 5

3. Product Requirements Document (PRD) . . . 6
3.1 Roles . . . 6
3.2 Core modules . . . 6
3.3 MVP boundary . . . 6
3.4 Functional requirements . . . 6
3.5 Non-functional requirements . . . 7

4. UX/UI Design System . . . 7
4.1 Information architecture . . . 7
4.2 Visual direction . . . 7
4.3 Interaction and accessibility rules . . . 8

5. Final Technology Stack . . . 8
5.1 Architecture choices . . . 9

6. System Architecture and Data Flow . . . 9
6.1 Typical photo upload flow . . . 9
6.2 Repository layout . . . 9

7. Database Design . . . 9
7.1 Database standards . . . 10

8. API Design . . . 10
8.1 Conventions . . . 10

9. Photo Identification and Visual Similarity . . . 11
9.1 Implementation sequence . . . 11
9.2 Processing and privacy controls . . . 11

10. Security Engineering and OWASP . . . 11
10.1 Minimum security controls . . . 12
10.2 Security delivery gates . . . 12
10.3 Threat scenarios to test . . . 12

11. Privacy, Trust, Safety and Content Rights . . . 13
11.1 Privacy by design . . . 13
11.2 Trust signals . . . 13
11.3 Reporting and moderation . . . 13

12. Quality Assurance and Release Acceptance . . . 13
12.1 Definition of done . . . 13

13. Deployment, Reliability and Scale . . . 14
13.1 Deployment baseline . . . 14
13.2 Observability . . . 14
13.3 Scale strategy . . . 14

14. Implementation Roadmap . . . 14
14.1 Recommended implementation order . . . 15

15. Project Budget and Resourcing . . . 15
15.1 Indicative monthly operating costs . . . 15
15.2 Roles to plan for . . . 16

16. Risk Register . . . 16

17. Governance and Decision Log . . . 16
17.1 Change-control process . . . 17

18. Glossary . . . 17

Final Recommendation . . . 17

Document Purpose and Scope

This document consolidates the product concept, business planning, requirements, UX direction, technology decisions, architecture, security controls, operations, budget assumptions and implementation sequence for M.A.T.E. It is intended to be a practical project baseline for the team, not a substitute for legal advice, vendor quotations, a production threat assessment or final engineering design.

Product naming note: this document consistently uses M.A.T.E. (Meeting Adventure Travel Escape). The previously mentioned “Nomigo” name is intentionally not used as the product name.

Subscription plans and pricing are deliberately excluded from the launch specification and can be added in a later commercial document.

How to use this document

- Product and business owners: confirm scope, target audience, launch geography and success metrics.
- Designers: use the UX principles and journeys to create tested screens and design-system components.
- Engineers: use the stack, API conventions, data model and architecture as the initial implementation baseline.
- QA and security: turn requirements into repeatable test cases and release gates.
- Operations: validate hosting, incident response, backups, moderation capacity and cost assumptions before launch.

Executive Summary

M.A.T.E. is a people-first social travel platform that helps users discover destinations, find compatible travel companions, connect, build travel plans, coordinate trips and share useful travel discoveries. Its core value is the relationship layer around travel—not simply bookings or a generic social feed.

The central product loop is: discover → build Travel DNA → find compatible travelers → connect → create a Travel Plan → coordinate in a Trip Room → travel → share useful memories and recommendations → inspire the next trip.

Recommended launch focus

- Launch with a focused set of destination communities rather than trying to cover every location at once.
- Make the Travel Plan, Travel DNA and companion matching experience the core of the product.
- Build a secure, accessible web experience first while keeping APIs and UI patterns ready for Flutter mobile apps.
- Ship reliable core workflows before introducing advanced AI features or expensive distributed infrastructure.
- Treat trust, safety, privacy and OWASP-aligned security as product requirements from the first sprint.

Decision area	Recommended baseline
Web	React.js + TypeScript + Vite + Tailwind CSS
Mobile	Flutter + Dart for Android and iOS
Backend	Node.js + Express.js + TypeScript modular monolith
Database	PostgreSQL
Media	Private object storage with controlled delivery
Photo similarity	SHA-256 exact duplicates, perceptual hashes, then optional image embeddings
Deployment	Docker, CI/CD, managed hosting and infrastructure as code
Security	OWASP Top 10, API Security Top 10, ASVS and MASVS-aligned checks

1. Project Proposal

1.1 Problem statement

Travelers often struggle to find people with compatible destinations, dates, budgets, interests and travel styles. Existing travel services are strong in bookings, maps, reviews or itinerary management, while broad social networks are not designed around the practical process of finding a travel companion and coordinating a trip. Useful local discoveries can also be scattered across posts, chats and personal notes.

1.2 Proposed solution

M.A.T.E. combines traveler profiles, structured travel preferences, destination discovery, companion matching, trip planning, group coordination and travel content in one connected experience. Matching should prioritize practical compatibility and explain the reasons for a suggested match rather than present an unexplained score.

1.3 Product objectives

- Help users find compatible travel companions based on destination, dates, budget and interests.
- Reduce friction from first discovery to a shared, actionable Travel Plan.
- Provide a structured Trip Room for itinerary, saved places, members and agreed safety information.
- Enable users to share destination guides, photos, hidden gems and trip memories.
- Build user trust with privacy controls, reporting, blocking, moderation and clear verification labels.
- Create a measurable loop that rewards successful connections and useful contributions—not merely scrolling time.

1.4 Target users

Segment	Needs / motivation	Initial acquisition channels
Students and young adults	Affordable trips, flexible dates, compatible groups	Campus clubs, student communities, ambassadors
Solo travelers	Companionship, confidence, shared interests	Travel communities, creator partnerships, referrals
Budget travelers	Budget-aware planning and cost transparency	Budget travel groups, destination guides, search content
Adventure and nature travelers	Activity fit, pace and destination interests	Hiking, photography and outdoor communities
Local explorers	Discover lesser-known places and share tips	Local creators, city communities and contributor programs
Experienced travelers	Find partners and preserve trip knowledge	Travel clubs, specialist communities and referrals

1.5 Value proposition

“Find people who travel like you, plan together, and turn shared discoveries into your next adventure.”

2. Business Plan and Go-to-Market

2.1 Positioning

M.A.T.E. should position itself as the relationship and coordination layer for social travel. It should complement mapping, booking, accommodation and itinerary services rather than claim to replace all of them.

Alternative category	Typical strength	M.A.T.E. differentiation
Maps and navigation	Routes, places, directions	Connect a place to compatible people and a shared trip plan

Alternative category	Typical strength	M.A.T.E. differentiation
Booking platforms	Accommodation and transport transactions	Help travelers decide who to travel with and how to coordinate
Review and guide platforms	Destination information and ratings	Combine discovery with personal travel preferences and group intent
Social networks	Reach, posting and messaging	Travel-specific profiles, plan context, safety controls and trip lifecycle
Itinerary tools	Organizing trip activities	Link itinerary planning to companion discovery and Trip Room membership

2.2 Customer acquisition strategy

- Start geographically or thematically narrow: select a few destinations and travel communities where a useful density of users can be built.
- Recruit campus ambassadors and community organizers; give them clear safety and conduct guidance.
- Partner with travel creators who can demonstrate a complete journey, not only a promotional post.
- Publish useful destination pages and hidden-gem content designed around real traveler questions.
- Create referral loops around a created Travel Plan or a successful group connection rather than raw invite counts.
- Run a closed beta and interview users who abandon onboarding, receive no matches or fail to complete a plan.
- Measure acquisition channel quality by activation, first meaningful connection, trip creation and repeat usage.

2.3 Launch phases

Phase	Approach	Exit signal
Private alpha	Team and invited testers validate core journeys	Critical flows work; no known high-risk access-control defect
Destination pilot	One or a few communities with curated content	Useful match density and repeated Travel Plan creation
Community launch	Campus, creator and local-community partnerships	Improving activation and connection acceptance
Regional expansion	Add destinations only with moderation and content coverage	Retention and safety operations remain healthy
Broader scale	Expand infrastructure based on observed load	Reliability, unit cost and support capacity meet targets

2.4 North Star and key metrics

North Star Metric: Successful Travel Connections—two or more users discover one another, connect, create a trip and report that they actually travelled together or completed the planned activity.

- Activation: registration-to-profile completion and time to first useful action.
- Matching: match-search success, match detail views and connection acceptance rate.
- Planning: Travel Plan creation, publishing and member-request conversion.
- Retention: repeat Travel Plan creation, repeat trip participation and return rate by destination cohort.
- Contribution: useful posts, saved places, guide completion and content reports/rejections.
- Safety: reports per active user, time to triage, block success, repeat offender rate and moderation backlog.
- Reliability: API error rate, latency, failed uploads, queue age and incident recovery time.

3. Product Requirements Document (PRD)

3.1 Roles

Role	Permissions and responsibilities
Guest	Browse public landing and selected public destination content.
Registered traveler	Manage own profile and Travel DNA, discover, connect, plan, share and report.
Trip creator	Manage plan settings, membership requests and permitted trip resources.
Trip member	Access member-only Trip Room features and contribute within assigned permissions.
Moderator	Review reports, apply approved content/account actions and record case outcomes.
Administrator	Manage system configuration and privileged operations with audit logging.

3.2 Core modules

- Authentication and account recovery.
- Profile, interests, languages and Travel DNA.
- Destination catalogue, search, filters and place details.
- Travel Plan creation, publishing, membership and lifecycle.
- Companion matching with hard filters and explainable compatibility.
- Connection requests, messaging and notifications.
- Trip Room, itinerary, saved places and safety information.
- Posts, photos, hidden gems, saves, comments and later guides/memories.
- Report/block workflows, moderation console and administrative audit trail.
- Media processing and photo duplicate/visual-similarity search.

3.3 MVP boundary

The first meaningful release should include sign-up/sign-in, profile and Travel DNA, destination discovery, Travel Plans, companion matching, connection requests, chat/notifications, Trip Room, posts/photos, hidden gems, report/block and basic verification signals. Rewards, advanced AI trip planning, automated memory generation, sophisticated vector search and commercial subscription plans can be staged later.

3.4 Functional requirements

ID	Requirement	Priority
AUTH-01	Users can register, sign in, recover access and log out securely.	MVP
PROF-01	Users can edit profile, interests, languages and travel preferences.	MVP
DEST-01	Destinations have canonical IDs, controlled categories and searchable attributes.	MVP
TRIP-01	Users can draft, publish, edit, cancel and archive Travel Plans.	MVP
TRIP-02	Plan capacity and membership transitions are enforced server-side.	MVP
MATCH-01	Candidates are filtered by destination/date context, privacy and block state before ranking.	MVP
MATCH-02	The interface explains meaningful compatibility factors without fabricating reasons.	MVP
MSG-01	Only authorized conversation participants can access messages.	MVP
CONT-01	Users can upload photos and create destination-linked posts subject to moderation.	MVP
SAFE-01	Users can report and block accounts/content from relevant surfaces.	MVP

ID	Requirement	Priority
MEDIA-01	Uploads are validated and processed asynchronously with clear status.	MVP
SIM-01	Exact duplicate detection is implemented before advanced visual similarity.	Phase 2

3.5 Non-functional requirements

- Performance: target p95 under 500 ms for common API reads at expected load, excluding media transfer and external provider latency; validate with tests.
- Availability: define service-level objectives before public launch and monitor critical journeys.
- Scalability: scale API replicas and media workers independently; use queues for retryable long-running tasks.
- Data integrity: use constraints and transactions for membership, connection and plan lifecycle transitions.
- Accessibility: target WCAG 2.2 AA for web where practical and equivalent mobile accessibility practices.
- Compatibility: define supported browsers and minimum Android/iOS versions.
- Recovery: define RPO/RTO and test backup restoration before production.
- Privacy: document purpose, visibility, retention, deletion and staff access for each sensitive data class.

4. UX/UI Design System

4.1 Information architecture

Area	Primary screens
Home	Personalized next steps, upcoming plans and relevant discovery
Explore	Destination search, categories, destination detail and guides
Find Travelers	Filters, match cards, compatibility reasons and profile details
Travel Plans	Plan list, create/edit flow, published detail and member requests
Trip Room	Overview, members, chat, itinerary, places, budget and safety
Share	Posts, stories, guides, hidden-gem submissions and memories
Messages	Conversation list, chat, connection state and reporting
Profile	Travel DNA, interests, travel history, visibility and settings
Admin	Moderation queue, reports, catalogue tools and audit history

4.2 Visual direction

- Use deep indigo/purple as the primary brand family, warm orange for selected emphasis, and neutral backgrounds with high-contrast text.
- Use a consistent spacing scale and reusable cards, chips, tabs, filters, buttons, alerts and form components.
- Keep destination and traveler cards image-led but make title, context and primary action immediately readable.
- Use restrained motion and avoid animation that delays important actions.
- Keep terminology consistent across web and mobile; adapt navigation to each platform rather than shrinking desktop screens.

4.3 Interaction and accessibility rules

- Every data-driven screen needs loading, success, empty, error, offline and permission-denied states where relevant.
- Forms preserve valid input when requests fail and explain how to recover.
- Use semantic headings, keyboard navigation, visible focus, screen-reader labels and scalable text.
- Do not rely on color alone for match status, verification or error states.
- Explain why sensitive information is requested and make optional fields clearly optional.
- Test the onboarding, matching, plan creation and report/block journeys with representative users before release.

5. Final Technology Stack

Use one primary technology for each responsibility. The stack below balances the user's selected technologies with a maintainable path to a large-scale platform.

Layer	Selected technology	Purpose
Web	React.js, TypeScript, Vite	Responsive browser client and typed component system
UI styling	Tailwind CSS + accessible component patterns	Reusable design tokens and responsive styling
Web routing	React Router	Navigation and route protection
Mobile	Flutter + Dart	Native Android/iOS application and camera/gallery integration
Backend API	Node.js + Express.js + TypeScript	REST API, domain rules, authorization and validation
API contract	OpenAPI 3.x	Documented endpoints and request/response schemas
Database	PostgreSQL	Transactional source of truth for core product records
Data access	Prisma ORM or reviewed parameterized SQL	Migrations and consistent database access; choose one approach
Authentication	OIDC/OAuth-capable identity provider	Sign-in, verification, token/session lifecycle and recovery
Media storage	Private S3-compatible object storage	Original images, thumbnails and derived media
Cache	Redis when measured useful	Hot reads, ephemeral state and distributed rate limiting
Jobs	Durable queue + worker processes	Media processing, notifications, indexing and retries
Image similarity	Python, OpenCV, perceptual hashes, image embeddings	Duplicate and visually similar photo discovery
AI service later	Python + FastAPI + PyTorch	Isolated model inference when justified
Packaging	Docker + Docker Compose	Repeatable local and deployable application environments
CI/CD	GitHub Actions or equivalent	Tests, scans, image builds and deployments
Infrastructure as code	Terraform	Reviewed and repeatable infrastructure configuration
Observability	OpenTelemetry + metrics/logging/error monitoring	Traceability, dashboards and actionable alerts
Security tools	CodeQL/Semgrep, dependency scanning, Trivy, OWASP ZAP	Static, dependency, container and dynamic security checks

5.1 Architecture choices

- Start with a modular monolith API organized by domain; extract services only when operational or ownership needs justify the complexity.
- Use PostgreSQL as the canonical store; do not store raw photo/video blobs inside relational rows.
- Use private object storage for media and keep storage keys and metadata in PostgreSQL.
- Begin with PostgreSQL search indexes and full-text search; add a dedicated search/vector engine when measured relevance or latency requires it.
- Keep Python image processing separate from the main API so model dependencies and CPU-heavy work do not destabilize request handling.
- Use managed container hosting first; Kubernetes is not an MVP requirement unless the team already has the operational expertise and a concrete need.

6. System Architecture and Data Flow

Layer	Responsibilities	Components
Experience	Screens, navigation, forms and interactions	React web, Flutter mobile
Edge	TLS, request routing, CDN/WAF and limits	Load balancer, CDN, WAF
Application	Authenticated product workflows and domain rules	Node.js/Express modular API
Async workers	Retryable or long-running work	Media validation, thumbnails, notifications, indexing
Data	Canonical records and media objects	PostgreSQL, private object storage, optional Redis
Intelligence	Matching and image similarity	Versioned scoring and Python/OpenCV workers
Operations	Logs, metrics, traces, alerts and audit	OpenTelemetry and monitoring stack

6.1 Typical photo upload flow

- Client requests a short-lived upload authorization; server validates identity, quotas and allowed media type.
- Client uploads directly to private object storage; backend stores a canonical media asset ID and processing state.
- A worker verifies the file signature, safely decodes the image, normalizes orientation and creates approved derivatives.
- The worker strips EXIF location metadata from public derivatives where appropriate and computes hashes.
- The asset becomes available only after required validation/moderation and access rules are satisfied.
- The API returns canonical IDs and permitted delivery URLs—not permanent public storage paths.

6.2 Repository layout

/apps/web React client; **/apps/mobile** Flutter app; **/services/api** Express API; **/services/media-worker** image jobs; **/services/ai** optional inference; **/packages/contracts** API schemas; **/infra** Docker/Terraform/deployment; **/docs** architecture decisions, OpenAPI, runbooks and threat model.

7. Database Design

Entity group	Representative records	Key rules
Identity/profile	users, profiles, travel_dna, interests, languages	User owns profile and preferences; visibility explicit
Destinations	destinations, places, destination_media, saved_destinations	Canonical destination/place IDs referenced by plans/content
Trips	travel_plans, plan_members, itinerary_items, saved_places, expenses	Explicit creator/member roles and lifecycle states

Entity group	Representative records	Key rules
Matching	match_runs, match_results, connections	Version score logic and record explainability inputs as needed
Messaging	conversations, conversation_members, messages	Access requires current membership and policy checks
Content	posts, stories, guides, hidden_gems, memories, comments, saves	Author, visibility, status and destination references
Media	media_assets, media_derivatives, media_similarity_index	Owner, storage key, hash, processing and retention state
Safety	blocks, reports, moderation_cases, verification_events	Restricted access and audit history
Operations	notifications, preferences, audit_logs, reward_events	Deduplication keys and event timestamps

7.1 Database standards

- Use stable non-guessable IDs for public-facing records and enforce foreign keys and unique/check constraints.
- Store timestamps in UTC; render them in the user's chosen/local context.
- Use transactions for joining a full plan, accepting a connection and other multi-row state transitions.
- Keep schema migrations version-controlled and test migration behavior against representative data.
- Use soft deletion only where product, retention or legal rules require it; define eventual hard deletion.
- Review indexes against actual query plans rather than indexing every field by default.
- Consider PostgreSQL Row-Level Security as defence in depth only with a coherent policy design; it does not replace application authorization.

8. API Design

8.1 Conventions

- Version APIs under a prefix such as /api/v1 and publish an OpenAPI specification.
- Use JSON for standard requests/responses and signed direct uploads for media.
- Validate every request on the server using explicit schemas.
- Use cursor pagination for large feeds and lists; impose maximum page sizes.
- Return consistent machine-readable error codes and safe human-readable messages.
- Use idempotency keys for retry-sensitive actions such as upload finalization or future payment flows.
- Apply timeouts, bounded retries and safe fallbacks to external providers.

Resource group	Example endpoints	Controls
Profile	GET/PATCH /api/v1/profile	Owner check and field-level validation
Travel DNA	GET/PATCH /api/v1/travel-dna	Validated preferences and visibility
Destinations	GET /api/v1/destinations	Pagination and search limits
Travel Plans	POST/GET /api/v1/travel-plans; PATCH /{id}; POST /{id}/publish	Lifecycle and membership validation
Matching	POST /api/v1/matches/search	Privacy/block filtering before ranking
Connections	POST /api/v1/connections; PATCH /{id}	Allowed state transitions only
Media	POST /api/v1/media/upload-intents; POST /api/v1/media/{id}/complete	Short-lived upload authorization and safe processing
Content/reporting	POST/GET /api/v1/posts; POST /api/v1/reports	Visibility and moderation enforcement

Resource group	Example endpoints	Controls
Admin	GET /api/v1/admin/reports	Privileged role, audit trail and restricted access

9. Photo Identification and Visual Similarity

M.A.T.E. should distinguish exact duplicate detection, near-duplicate detection and visual similarity. These are different functions and must not be presented as equally certain. This requirement concerns images and scenes, not identifying a person by their face.

Mode	Method	What it finds / limitation
Exact duplicate	SHA-256 checksum of file bytes	Identical files; resizing or recompression changes the checksum.
Near duplicate	Perceptual hash (pHash/dHash/aHash) and Hamming distance	Often finds resized/recompressed copies; heavy crop, overlay or rotation can reduce accuracy.
Visual similarity	Vision-model embeddings and nearest-neighbour retrieval	Finds similar visual content; similar-looking images are not necessarily the same photo or place.
Same-place suggestion (later)	Embeddings plus user-provided place metadata and optional geo context	Only a suggestion; image appearance alone cannot establish location with certainty.

9.1 Implementation sequence

Stage	Implementation	Exit criteria
1	SHA-256 exact duplicate checks	Known duplicate test set passes; duplicate upload behavior is clear
2	Perceptual hashes and tunable thresholds	Test resized, compressed, crop and rotation cases; false positives reviewed
3	Python/OpenCV plus image embeddings and vector index if needed	Offline precision/recall and latency meet agreed targets
4	Scale workers and index infrastructure	Load tests establish throughput, queue behavior and cost per image

9.2 Processing and privacy controls

- Validate file signatures, dimensions, size and decoder behavior; never trust file extensions alone.
- Run image decoding in constrained workers with CPU, memory, time and network limits.
- Store hash/model version/processing status to support reproducibility and reprocessing.
- Filter similarity results by access permissions before returning candidates.
- Propagate deletion to thumbnails, hashes, embeddings and search indexes according to the retention policy.
- Do not expose hidden EXIF location data; strip metadata from public derivatives where appropriate.
- Evaluate precision@k, recall@k, false-positive rate, latency and cost per image; do not promise perfect accuracy.

10. Security Engineering and OWASP

Security must cover the web client, mobile apps, API, database, media pipeline, background workers, AI services and cloud infrastructure. Maintain a threat model and treat security verification as a release requirement.

Standard / practice	M.A.T.E. application
OWASP Top 10	Baseline review for access control, injection, configuration, cryptographic failures and vulnerable components.

Standard / practice	M.A.T.E. application
OWASP API Security Top 10	Test broken object-level authorization, authentication, property-level access, resource consumption and function-level permissions.
OWASP ASVS	Select a risk-appropriate verification baseline and turn requirements into test cases.
OWASP MASVS	Review mobile local storage, network communication, authentication, platform interaction and build/release controls.
OWASP SAMM	Improve secure design, implementation, verification and operations maturity over time.
OWASP ZAP	Run controlled dynamic scans in staging, including authenticated scans where practical.
SAST / dependency scanning	Use CodeQL or Semgrep and dependency vulnerability alerts with lockfile review.
Container scanning	Use Trivy or equivalent for vulnerable packages and unsafe container configuration.
Secrets scanning	Prevent credentials and private keys entering source control or build artifacts.

10.1 Minimum security controls

- Authentication: use a reputable identity provider or reviewed session implementation; require strong controls for privileged accounts.
- Authorization: check ownership, membership, role, visibility, block state and resource state on every sensitive operation.
- Input handling: validate schemas, use parameterized queries and safely render user-generated content.
- Rate limits: protect sign-in, recovery, search, connection requests, messaging, uploads and similarity queries.
- Secrets: store server credentials in a managed secret store; never embed privileged keys in web/mobile bundles.
- Transport/storage: enforce TLS, encrypt storage and backups, and use least-privilege database roles.
- Audit logs: record privileged and safety-relevant actions without logging passwords, tokens or unnecessary private content.
- Abuse controls: detect spam, scraping, account takeover, unwanted contact and expensive request flooding.
- Recovery: maintain tested backups, incident contacts, rollback procedures and vulnerability response workflows.

10.2 Security delivery gates

Stage	Required checks
Pull request	Lint, type checks, unit tests, secret scan, SAST and dependency scan
Build	Locked dependencies, container scan and artifact provenance where available
Staging	Integration tests, IDOR/BOLA tests, ZAP scan, upload security and rate-limit tests
Release	No unresolved critical vulnerability; high risks fixed or formally accepted by accountable owner
Production	Alerting, audit logs, backup verification, incident response and rollback readiness

10.3 Threat scenarios to test

- A user changes a plan ID to access another group’s private Trip Room.
- A blocked user attempts to contact or discover the blocker through an alternate route.
- A client changes owner_id, role, verification status or reward points in a request body.
- An upload uses a fake extension, malformed image or excessive dimensions.
- A user floods connection requests or repeatedly invokes expensive image similarity searches.
- A deleted post remains accessible via a thumbnail, CDN cache or search index.
- An admin account is compromised or an internal moderation tool exposes reports to unauthorized staff.

11. Privacy, Trust, Safety and Content Rights

11.1 Privacy by design

- Explain the purpose of profile fields, Travel DNA and location-related permissions.
- Make public, connections-only and private visibility choices explicit where supported.
- Use precise location only when needed, with opt-in consent and an obvious stop-sharing action.
- Define retention and deletion behavior for accounts, media, messages, backups and derived image indexes.
- Restrict staff access to reports, verification records and sensitive trip details.
- Obtain legal review of privacy notices, terms, age eligibility, consent and data protection requirements in launch markets.

11.2 Trust signals

Potential trust signals include email/phone verification, completed-trip history, references and user feedback. Each badge must state what was checked and when. Verified email does not prove identity, and a rating cannot guarantee safety. Do not market any score or badge as a safety guarantee.

11.3 Reporting and moderation

- Allow reports from profile, message, post and plan surfaces.
- Confirm receipt without revealing reporter identity to the reported user.
- Triage by severity and maintain case status, evidence access controls and action history.
- Provide block controls independently of the report review.
- Create a copyright complaint/takedown workflow and a dispute or appeal path.
- Define policies for harassment, scams, impersonation, private information and unsafe or misleading advice.

12. Quality Assurance and Release Acceptance

Test level	Coverage
Unit	Matching logic, validators, permissions and plan state transitions
Integration	Database transactions, storage authorization, queues and provider failures
Contract	OpenAPI schemas, error formats and client/server compatibility
End-to-end	Sign-up → Travel DNA → plan → match → connection → conversation → Trip Room → share
Security	IDOR/BOLA, role escalation, session handling, injection, upload attacks and rate limits
Performance	API load, database contention, pagination, media throughput and queue backlog
Mobile	Camera/gallery permissions, app backgrounding, retries, deep links and push notifications
Accessibility	Keyboard/screen-reader checks, contrast, text scaling, focus order and error announcements

12.1 Definition of done

- Acceptance criteria and edge cases are covered by tests.
- Authorization is tested at the API/data boundary, not only in the UI.
- Loading, empty, error and permission-denied states are implemented.
- Monitoring, support guidance and rollback steps exist for production-impacting changes.
- Documentation, API contracts, migrations and tests are updated together.
- No known cross-user data exposure remains in profiles, plans, conversations, media or reports.
- Backup restore and deployment rollback are tested, not merely documented.

13. Deployment, Reliability and Scale

13.1 Deployment baseline

- Package API and workers as Docker images; separate configuration and credentials by environment.
- Use managed PostgreSQL with automated backups and point-in-time recovery where available.
- Use private object storage with lifecycle policies and controlled delivery.
- Use CDN delivery for approved public derivatives; private assets require authorization or short-lived signed URLs.
- Promote the same tested build artifact between test, staging and production.
- Use feature flags for controlled rollout and fast disablement of unstable features.

13.2 Observability

Signal	Examples
Logs	Request ID, route, outcome and latency; redact secrets and sensitive content
Metrics	Request rate, error rate, p95/p99 latency, DB connections, queue age and worker restarts
Traces	Follow API work across database, queue and external dependencies
Alerts	Critical journey failures, DB saturation, failed backups, queue backlog and suspicious auth activity
Product health	Onboarding failures, empty match searches, plan publishing errors and report backlog

13.3 Scale strategy

- Scale stateless API replicas and media workers independently.
- Profile slow endpoints and query plans before increasing database size.
- Introduce Redis only for measured caching or distributed coordination needs.
- Add dedicated search/vector infrastructure when latency or relevance data justifies it.
- Extract a domain service only when independent deployment, ownership or reliability benefits outweigh the added operational burden.
- Define recovery point objective (RPO) and recovery time objective (RTO), then test restore procedures regularly.

14. Implementation Roadmap

The timeline below is an indicative planning baseline, not a delivery guarantee. Actual duration depends on team size, current code quality, provider approvals and the final MVP boundary.

Phase	Indicative duration	Deliverables / exit gate
0. Discovery and audit	1–2 weeks	Review current source documents/code, user needs, MVP and risk register
1. Foundation	2–3 weeks	Repository, environments, CI, UI tokens, API skeleton, migrations and monitoring
2. Auth and profile	2–4 weeks	Sign-in, profile, Travel DNA, privacy controls and authorization tests
3. Destinations and plans	3–4 weeks	Destination catalogue, plan lifecycle and membership rules
4. Matching and connections	3–5 weeks	Candidate filters, versioned scoring, explanations and connection states
5. Messaging and Trip Room	3–5 weeks	Authorized conversations, notifications, itinerary, saved places and safety details
6. Content and moderation	3–5 weeks	Posts/photos, hidden gems, report queue and admin workflows
7. Flutter mobile	4–8 weeks	Android/iOS app, camera/gallery, push and API integration

Phase	Indicative duration	Deliverables / exit gate
8. Hardening and pilot	2–4 weeks	Performance, OWASP checks, backup restore, analytics and closed beta
9. Photo similarity	2–5 weeks after media MVP	Exact duplicate first; near-duplicate/embeddings after evaluation
10. AI and scale evolution	Ongoing	Optional trip/content assistance and measured infrastructure changes

14.1 Recommended implementation order

- Audit and stabilize any existing working functionality before major rewrites.
- Build one end-to-end workflow at a time; do not stop at isolated screens without API, data and test coverage.
- Keep database changes backward-compatible during rolling deployments where practical.
- Build and validate core web journeys early, then implement mobile-specific behaviors in Flutter.
- Introduce photo similarity only after upload authorization, deletion, moderation and media processing are reliable.
- Add AI features only after core product metrics and operational controls are in place.

15. Project Budget and Resourcing

The following are directional planning ranges in Indian rupees, not vendor quotations. Confirm scope, staffing, existing code, taxes and cloud usage before approval.

Workstream	Indicative one-time range	Scope
Discovery, UX and architecture	₹1.5L–₹4L	Research, workflows, prototypes, architecture and threat model
React web client	₹3L–₹8L	Responsive screens, forms and core journeys
Backend and database	₹5L–₹12L	API modules, schema, permissions and domain logic
Flutter mobile	₹4L–₹10L	Android/iOS, camera/gallery, notifications and mobile UX
Core travel modules	₹4L–₹10L	Matching, plans, Trip Room, content, moderation and admin
QA and security	₹2L–₹6L	Automation, OWASP checks, performance and release hardening
DevOps and deployment	₹1.5L–₹5L	CI/CD, environments, monitoring, backups and runbooks
Contingency	10–20%	Integration risk, scope clarification and rework

A preliminary production-build envelope may be around ₹22 lakh–₹65 lakh or more before taxes and ongoing operations. A lean MVP may cost less if scope is reduced and later modules are deferred.

15.1 Indicative monthly operating costs

Category	Indicative range/month	Main driver
App/API hosting	₹10,000–₹60,000	Traffic, redundancy and compute
Managed PostgreSQL	₹5,000–₹40,000+	Storage, backup, availability and query load
Object storage/delivery	₹2,000–₹50,000+	Media volume, retention and egress
Queue/cache/monitoring	₹3,000–₹30,000+	Service tier, event volume and log retention
Email/SMS/push	₹1,000–₹25,000+	Verification and notification volume
Security/backup tools	₹2,000–₹25,000+	Vendor selection and retention
AI/GPU inference later	₹0–₹1,50,000+	Model, throughput and latency needs

15.2 Roles to plan for

- Product owner / product manager and UX/UI designer.
- React/TypeScript frontend engineer and Node.js/Express backend engineer.
- Flutter engineer for Android/iOS.
- QA automation engineer and DevOps/security engineering support.
- Community operations and trained moderation capacity for the pilot.

16. Risk Register

Risk	Potential impact	Mitigation
Low user density	Few relevant matches and low retention	Launch by destination/community; seed useful content and measure empty searches
Harassment or unsafe interactions	User harm and trust loss	Report/block, moderation playbook, safety education and escalation
Privacy/authorization leak	Sensitive data exposure	Threat model, least privilege, IDOR tests, audit logs and release gates
Copyright disputes	Content removal or legal exposure	Upload rights guidance, takedown workflow and appeal path
Poor match relevance	Users stop using matching	Explain factors, collect feedback and evaluate versioned weights
High media cost	Operating cost growth	Upload limits, derivatives, lifecycle rules and cost dashboards
Premature architecture complexity	Slower delivery and operational burden	Modular monolith first; add services only for measured reasons
AI misinformation	Misleading travel advice	Label assistance, use bounded sources and require user review before publication
Scope creep	Budget and schedule overruns	Approved MVP, backlog owner and documented change control
Moderation backlog	Unsafe content remains visible	Severity-based queues, staffing plan and escalation metrics

17. Governance and Decision Log

Decision	Current baseline	Revisit when
Mobile framework	Flutter	User testing or team capability shows material issue
Backend architecture	Node.js/Express modular monolith	Independent scaling/ownership needs justify extraction
Primary database	PostgreSQL	Measured workload requires complementary store
Media storage	Private object storage + canonical metadata	Traffic, compliance or delivery needs change
Photo matching	SHA-256 → perceptual hash → optional embeddings	Evaluation demonstrates value and demand
Search	PostgreSQL initially	Latency/relevance tests justify dedicated search
Security	OWASP-aligned design, CI and release checks	Risk profile or regulatory scope changes
Subscriptions	Not specified in this document	Separate commercial strategy is approved

17.1 Change-control process

- Record each significant decision, rationale, alternatives, owner and date.
- Update requirements, UI flows, API contracts, schema and test cases together.
- Assess migration, security, privacy, cost and support impacts before implementation.
- Maintain a deferred-work list so later features are not mistaken for MVP commitments.

18. Glossary

Term	Meaning
API	Application Programming Interface between clients and backend services
Travel DNA	Structured travel preferences used for personalization and companion matching
Travel Plan	Trip-intent record with destination, dates, budget, activities and group settings
Trip Room	Member-controlled workspace attached to a Travel Plan
RBAC	Role-Based Access Control
OWASP	Open Worldwide Application Security Project
ASVS / MASVS	Application Security Verification Standard / Mobile Application Security Verification Standard
CI/CD	Continuous Integration / Continuous Delivery
Object storage	File storage for media separate from relational database rows
Perceptual hash	Compact image representation used to detect visually similar image versions
Embedding	Numeric vector representation used to retrieve visually similar content
RPO / RTO	Recovery Point Objective / Recovery Time Objective
MVP	Minimum Viable Product

Final Recommendation

Build M.A.T.E. as a secure, modular social travel platform centered on Travel DNA, destinations, explainable companion matching, Travel Plans and the Trip Room. Use React.js and TypeScript for web, Flutter for Android/iOS, Node.js with Express.js for the API, PostgreSQL for core records, private object storage for media, Docker for packaging and OWASP-aligned DevSecOps from the first sprint. Start photo matching with exact duplicate checks, then add perceptual hashes and visual embeddings only after evaluation demonstrates value.

For launch, focus on a small number of destination communities, seed useful destination content, work with campus and travel communities, and measure successful travel connections. Subscription plans and pricing remain out of scope until a separate commercial strategy is approved.

Planning disclaimer: scope, budget, timeline, legal requirements, infrastructure sizing and security targets must be confirmed by the accountable project team before production commitments.